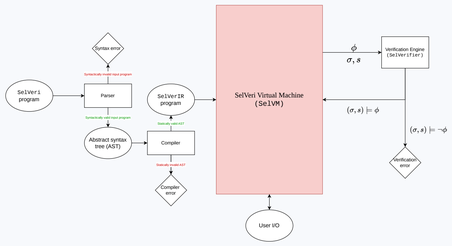
SelVeri

Self-Verifying Programming Language

Yağız Kaan Aydoğdu · Yusuf Demir | Advisor: Doğan Ulus

Project Overview

- SelVeri is an educational imperative language that **connects sequential programming with formal methods**.
- Programs may contain **first-order logic** or **linear temporal logic** specifications that are checked dynamically as execution reaches verification points.
- The implementation parses high-level programs, compiles into the intermediate representation, which is executed on a stack-based abstract machine.
- The verification engine translates runtime configurations to mathematical constraints and uses them to verify logical specifications.



Syntax

$$
\begin{aligned}
P &::= F^*\ S^* \\
F &::= \texttt{function}\ f(\overline{x : T}) \to T :: S^*\ \texttt{end} \\
T &::= \tau \mid \texttt{List}[\tau, d, \overline{A}], \qquad \tau ::= \texttt{Int} \mid \texttt{Real} \\
S &::= x : T \mid x := A \mid x[A] := A \mid \texttt{pass} \mid \texttt{call}\ f(\overline{A}) \\
&\quad \mid \texttt{write}(A) \mid \texttt{writeline}(A) \mid \texttt{return}\ A \mid \{\mathit{Spec}\} \\
&\quad \mid \texttt{if}\ B\ \texttt{then}\ S^*[\texttt{else}\ S^*]\ \texttt{fi} \\
&\quad \mid \texttt{while}\ B\ \texttt{do}\ S^*\ \texttt{od} \\
A &::= n \mid r \mid x \mid [\overline{A}] \mid A \oplus A \mid -A \mid x[A] \mid \texttt{len}(x) \mid \texttt{read}() \\
&\quad \mid \texttt{obtain}(\&x, \mathit{Spec}) \\
B &::= \texttt{true} \mid \texttt{false} \mid A \bowtie A \mid \texttt{not}\ B \mid B \circ B \mid \{\mathit{Spec}\}
\end{aligned}
$$

Simplified syntax of SelVeri in Backus-Naur Form.

- Actual syntax is more sophisticated to impose implementation details, such as operator precedence.

Semantics

Configuration

$$\gamma = \langle S, \sigma, s, \iota, \Omega \rangle$$

- σ program state, mapping variables to values
- s scope, mapping variables to declared types
- ι input stream consumed by `read()`
- Ω output and verification trace

Small-Step Rules

$$
\begin{aligned}
\langle x : T, \sigma, s \rangle &\to \langle \sigma[x \mapsto \mathbf{0}_T],\ s[x \mapsto T] \rangle \\
\langle x := A, \sigma, s \rangle &\to \langle \sigma[x \mapsto \mathcal{A}(A, \sigma, s, \iota)],\ s \rangle \\
\langle \texttt{if}\ b\ \texttt{then}\ S_1\ \texttt{else}\ S_2, \sigma, s \rangle &\to \begin{cases} \langle S_1, \sigma, s \rangle & \mathcal{B}(b, \sigma, s) = 1 \\ \langle S_2, \sigma, s \rangle & \mathcal{B}(b, \sigma, s) = 0 \end{cases} \\
\langle \texttt{while}\ b\ \texttt{do}\ S, \sigma, s \rangle &\to \begin{cases} \langle S; \texttt{while}\ b\ \texttt{do}\ S, \sigma, s \rangle & \mathcal{B}(b, \sigma, s) = 1 \\ \langle \sigma, s \rangle & \mathcal{B}(b, \sigma, s) = 0 \end{cases}
\end{aligned}
$$

- $\mathcal{A}$ evaluates arithmetic expressions; $\mathcal{B}$ evaluates boolean guards as 1 or 0.
- Declarations extend scope; assignments update state; undefined configurations enter erroneous program state, $\hat{\sigma}$.

Compilation to SelVerIR

$$\mathcal{C} : \mathbf{Code}_{\text{SelVeri}} \to \mathbf{Code}_{\text{IR}},\ \mathcal{CA} : \mathbf{AExp} \to \mathbf{Code}_{\text{IR}},$$
$$\mathcal{CB} : \mathbf{BExp} \to \mathbf{Code}_{\text{IR}}$$

SelVeri fragment	SelVerIR emitted
a	$\mathcal{CA}(a)$ leaves the value of a on stack e
b	$\mathcal{CB}(b)$ leaves 1 or 0 on stack e
$x : T$	`DECL` T x
$x := a$	$\mathcal{CA}(a)$: `STORE` x
$S_1; S_2$	$\mathcal{C}(S_1) : \mathcal{C}(S_2)$
`if` b `then` S_1 `else` S_2	$\mathcal{CB}(b)$: `JZ` L_e : $\mathcal{C}(S_1)$: `GOTO` L_o : L_e : $\mathcal{C}(S_2)$: L_o
`while` b `do` S	L_h : $\mathcal{CB}(b)$: `JZ` L_o : $\mathcal{C}(S)$: `GOTO` L_h : L_o

- The compiler emits labeled stack-machine commands; labels become concrete program counters after jump patching.

SelVeri program

```
x : Int;
x := 3 + 4;
write(x);
```

$\Longrightarrow$

SelVerIR program

```
DECL INT x
PUSH 3 : PUSH 4 : ADD : STORE x
LOAD x : WRITE
```

Logic Programming Extension: SelVeri LP

- SelVeri LP **closes the gap between imperative programming and logic/declarative programming**.
- Specifications can act as boolean guards in ordinary control flow:

```
if { Historically y >= x + z => b != 0 } then
  b := 1;
else
  b := 0;
fi
```

- `obtain` asks Z3 to extract a witness satisfying a logical specification, then stores it in program state:

```
x := obtain(&x, ∃ y ∈ Int. &y = n*n*n)
```

Proven Properties of SelVeri

Correctness of Compilation

Expressions	structural induction on the syntactic structure of expressions
Terminating	strong induction on execution length
Divergent	transfinite induction on exectuion length while utilizing the ω-bound of computation

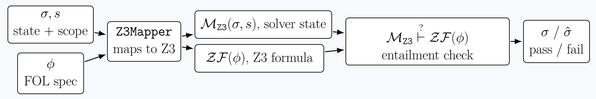
Turing complete	SelVeri can represent every general (μ)-recursive function
Sound verifier	accepted encodings satisfy the source specification
Deterministic	each valid configuration has at most one next state

Verified compilation transfers Turing-completeness and determinism to SelVerIR.

FOL Verification with Z3 Theorem Prover

- `Z3Mapper` maps the runtime configuration and FOL specification into equivalent Z3 inputs.
- `free_var_map` resolves program variables; `bound_var_map` resolves quantified variables.

`VERI` ϕ succeeds iff $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi)$

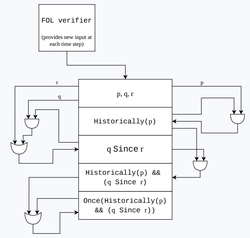


Past-LTL Verification

- Past-LTL specifications are evaluated over the finite history of runtime snapshots.
- SelVerifier stores memoized truth values for temporal subformulae, forming a **sequential network over time**.

```
Once(Historically p
  && (q Since r))
```

$\Rightarrow$

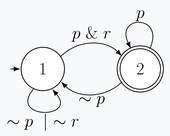


Future-LTL Verification

- fLTL specifications are translated to **deterministic finite automata** (DFA) with `ltl2dfa` library.
- At each step, Z3 evaluates the propositions (FOL); the DFA consumes the valuation and advances state.

```
Eventually((Always(p)
  && (q Until r)))
```

$\Rightarrow$



Separated-LTL Verification

- **Gabbay's separation theorem** states that any arbitrary mixed LTL formula can be rewritten in the form $\varphi_{past} \Rightarrow \varphi_{future}$ where φ_{past} contains only past operators and φ_{future} contains only future operators.
- Although SelVeri does not support the verification of mixed-LTL specifications, it is able to verify sLTL formulae to ensure the completeness of verification.

Algorithm `verify_sLTL`($\varphi_{past} \Rightarrow \varphi_{future}$):
  if `verify_pLTL`(φ_{past}) **then**
    return `verify_fLTL`(φ_{future})
  return `true` *// vacuous truth*

Future Work

- **Better user feedback:** Counter-example generation for failed specifications, failure recovery during parsing, and more informative error messages for undefined behavior.
- **Faster checks:** Dependency-aware LTL verification and definite accept state detection for DFAs, more optimized (possibly incremental) SMT queries, more efficient encoding of runtime configurations, and multi-threading for Verification Engine.
- **Language features:** More basic types (such as `String`), new dependent types (such as `Pair<t1,t2>`), and anything else a user can expect from a modern programming language.
- **Idiomatic formalization:** SelVeri's formalization, although strong, has been done with pen and paper. A more rigorous mechanized formalization in a proof assistant such as Lean or Isabelle would be a valuable future contribution.

How to Use SelVeri on Your Computer

- Install the command-line tool with `pip install selveri`.
- Write and run `.svi` programs locally from a terminal with `selveri <program_name>.svi`.
- You may use `-emit-ir` flag to see the generated intermediate code (`.svir`) to skip compilation and run directly on the IR virtual machine with `selveri -run-ir <program_name>.svir`.
- Consider installing the **SelVeri VS Code extension** for editor support such as syntax highlighting, mathematical notation support and language tooling.

References

1. K. R. R. Leino, "Dafny: An Automatic Program Verifier for Functional Correctness," 2010.
2. D. J. Pearce, "Whiley: a Platform for Research in Software Verification," 2013.
3. G. T. Leavens, A. L. Baker, and C. Ruby, "Preliminary Design of JML," 2006.
4. H. R. Nielson and F. Nielson, *Semantics with Applications: An Appetizer*, 2007.
5. L. de Moura and N. Bjørner, "Proofs and refutations, and Z3," 2008.
6. K. Havelund and G. Roşu, "Efficient monitoring of safety properties," 2004.
7. O. Lichtenstein, A. Pnueli, and L. Zuck, "The glory of the past," 1985.
8. G. De Giacomo and M. Y. Vardi, "Linear Temporal Logic and Linear Dynamic Logic on Finite Traces," 2013.
9. J. G. Henriksen et al., "Mona: Monadic second-order logic in practice," 1995.
10. D. M. Gabbay, "Declarative Past and Imperative Future," 1989.
11. D. M. Gabbay, I. Hodkinson, and M. Reynolds, *Temporal Logic*, Vol. I, 1994.